



Project Report

Database Management System (CT-261)

Project Name: E-commerce

Group Members:

Ayan Adnan (DT-22014)
Shehroz Aijaz (DT-22011)
Aman Qazi (DT-22044)
Awais Waseem (DT-22033)

Course Instructor:

Sir Rohail Qamar

Abstract

This project report details the development of an internal e-commerce management system using a relational database and a Flask web application. The system is designed to facilitate efficient oversight and management of key business components, including products, sellers, customers, and orders.

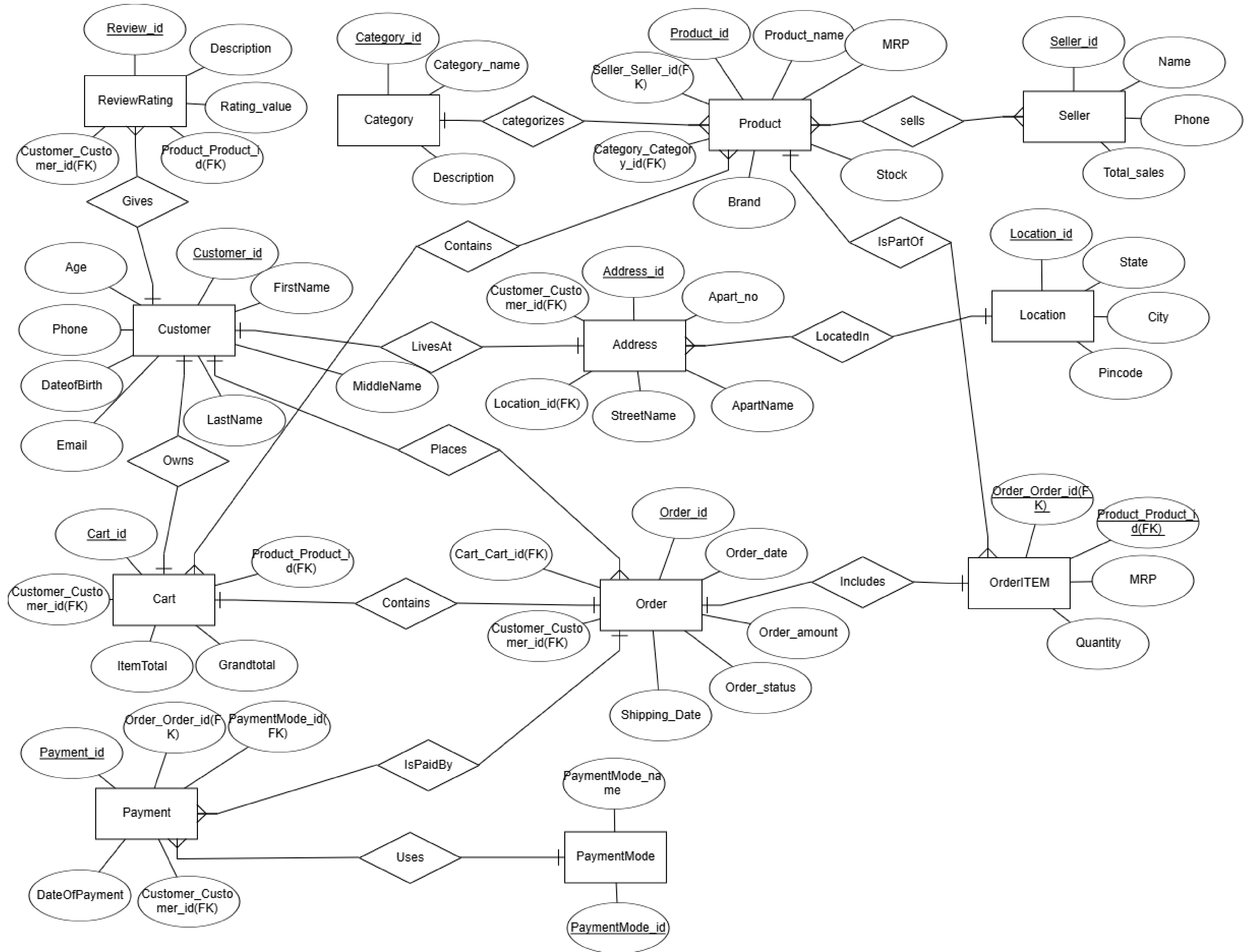
The database schema is constructed with well-defined tables for Categories, Sellers, Products, Customers, Addresses, Carts, Orders, Order Items, Reviews, Payments, and Ratings. Relationships between these entities are established using foreign key constraints to ensure data integrity and facilitate comprehensive data management.

A Flask-based web application serves as the interface, allowing management personnel to view and manipulate data within the system. Key features include:

- **Product Management:** View, update, and delete product details, including pricing, stock status, and associated category and seller information.
- **Seller Management:** Monitor seller information and sales data, with options for updates and deletions.
- **Customer Management:** Access detailed customer profiles, including contact information and order history.
- **Order Management:** Track and manage orders, from placement to delivery, ensuring efficient order processing and fulfillment.

This system streamlines administrative tasks and enhances data accessibility for management, ensuring operational efficiency and effective decision-making. The combination of a robust database structure and a user-friendly Flask application provides a reliable solution for managing an e-commerce platform's backend operations.

ER Diagram



DDL Queries

-- Create the Order table

```
CREATE TABLE "Order" (  
    Order_id SERIAL PRIMARY KEY,  
    Order_date TIMESTAMP NOT NULL,  
    Order_amount FLOAT NOT NULL,  
    Shipping_Date TIMESTAMP,  
    Order_status VARCHAR NOT NULL,  
    Cart_Cart_id INT,  
    Customer_customer_id INT,  
    FOREIGN KEY (Cart_Cart_id) REFERENCES Cart(Cart_id),  
    FOREIGN KEY (Customer_customer_id) REFERENCES Customer(Customer_id)  
);
```

-- Create the Customer table with Phone as VARCHAR

```
CREATE TABLE Customer (  
    Customer_id SERIAL PRIMARY KEY,  
    FirstName VARCHAR NOT NULL,  
    MiddleName VARCHAR,  
    LastName VARCHAR,  
    Email VARCHAR NOT NULL,  
    DateOfBirth DATE,  
    Phone VARCHAR,  
    Age INT  
);
```

-- Create the Payment table

```
CREATE TABLE Payment (  
    Payment_id SERIAL PRIMARY KEY,  
    ORDER_Order_id INT,  
    PaymentMode VARCHAR,  
    Customer_Customer_id INT,  
    DateOfPayment TIMESTAMP,  
    FOREIGN KEY (ORDER_Order_id) REFERENCES "Order"(Order_id),  
    FOREIGN KEY (Customer_Customer_id) REFERENCES Customer(Customer_id)  
);
```

DML Queries

```
INSERT INTO "Order" (Order_date, Order_amount, Shipping_Date, Order_status, Cart_Cart_id, Customer_customer_id) VALUES
('2024-06-10 14:30:00', 699.99, '2024-06-12 09:00:00', 'Shipped', 1, 1),
('2024-06-11 10:20:00', 49.99, '2024-06-13 11:00:00', 'Processing', 2, 2),
('2024-06-11 15:45:00', 19.99, '2024-06-14 16:00:00', 'Delivered', 3, 3),
('2024-06-12 09:00:00', 129.99, '2024-06-15 12:00:00', 'Pending', 4, 4);
```

```
INSERT INTO Customer (FirstName, MiddleName, LastName, Email, DateOfBirth, Phone, Age) VALUES
('John', 'A.', 'Doe', 'john.doe@example.com', '1985-08-15', '1234567890', 38),
('Jane', 'B.', 'Smith', 'jane.smith@example.com', '1990-01-20', '2345678901', 34),
('Emily', 'C.', 'Jones', 'emily.jones@example.com', '1995-05-25', '3456789012', 29),
('Michael', 'D.', 'Brown', 'michael.brown@example.com', '1988-12-10', '4567890123', 36);
```

```
INSERT INTO Payment (ORDER_Order_id, PaymentMode, Customer_Customer_id, DateOfPayment) VALUES
(1, 'Credit Card', 1, '2024-06-10 15:00:00'),
(2, 'PayPal', 2, '2024-06-11 11:00:00'),
(3, 'Credit Card', 3, '2024-06-11 16:00:00'),
(4, 'Debit Card', 4, '2024-06-12 10:00:00');
```

```
-- Insert data into Seller table (unchanged)
INSERT INTO Seller (Name, Phone, Total_sales) VALUES
('TechStore', '123-456-7890', 50000.00),
('FashionHub', '987-654-3210', 30000.00),
('BookWorld', '555-555-5555', 20000.00),
('HomeEssentials', '444-444-4444', 15000.00);
```

```
-- Insert data into Product table (unchanged)
INSERT INTO Product (Product_name, MRP, Stock, Brand, Category_CategoryID, Seller_Seller_id) VALUES
('Smartphone', 699.99, TRUE, 'BrandX', 1, 1),
('Jeans', 49.99, TRUE, 'BrandY', 2, 2),
('Cooking Book', 19.99, TRUE, 'BrandZ', 3, 3),
('Microwave Oven', 129.99, TRUE, 'BrandA', 4, 4);
```

```
|
```

Join Queries

Detailed Order Information with Customer and Product Details:

QueryQuery History

1234567891011121314151617181920212223

SELECT

o.Order_id,

o.Order_date,

o.Order_amount,

o.Shipping_Date,

o.Order_status,

c.FirstName,

c.LastName,

c.Email,

p.Product_name,

p.Brand,

oi.MRP,

oi.Quantity

FROM

"Order" o

JOIN

Customer c ON o.Customer_customer_id = c.Customer_id

JOIN

OrderITEM oi ON o.Order_id = oi.Order_Order_id

JOIN

Product p ON oi.Product_product_id = p.Product_id

WHERE

o.Order_id = 1;

Data OutputMessagesNotifications

	order_id integer	order_date timestamp without time zone	order_amount double precision	shipping_date timestamp without time zone	order_status character varying	firstname character varying	lastname character varying	email character varying	product_name character varying	brand
1	1	2024-06-10 14:30:00	699.99	2024-06-12 09:00:00	Shipped	John	Doe	john.doe@example.com	Smartphone	BrandX

Retrieve Customer Orders with Payment Information:

QueryQuery History

1234567891011121314

SELECT

p.Product_id,

p.Product_name,

p.MRP,

p.Stock,

p.Brand,

s.Name AS Seller_Name,

s.Phone AS Seller_Phone,

s.Total_sales AS Seller_Total_Sales

FROM

Product p

JOIN

Seller s ON p.Seller_Seller_id = s.Seller_id;

Data OutputMessagesNotifications

	product_id integer	product_name character varying	mrp double precision	stock boolean	brand character varying	seller_name character varying	seller_phone character varying	seller_total_sales double precision
1	1	Smartphone	699.99	true	BrandX	TechStore	123-456-7890	50000
2	2	Jeans	49.99	true	BrandY	FashionHub	987-654-3210	30000
3	3	Cooking Book	19.99	true	BrandZ	BookWorld	555-555-5555	20000
4	4	Microwave Oven	129.99	true	BrandA	HomeEssentials	444-444-4444	15000

Retrieve Orders with Product Details:

```
1 SELECT
2     o.Order_id,
3     o.Order_date,
4     o.Order_amount,
5     o.Order_status,
6     p.Product_id,
7     p.Product_name,
8     p.Brand,
9     oi.MRP,
10    oi.Quantity
11 FROM
12     "Order" o
13 JOIN
14     OrderITEM oi ON o.Order_id = oi.Order_Order_id
15 JOIN
16     Product p ON oi.Product_product_id = p.Product_id;
```

	order_id integer	order_date timestamp without time zone	order_amount double precision	order_status character varying	product_id integer	product_name character varying	brand character varying	mrp double precision	quantity integer
1	1	2024-06-10 14:30:00	699.99	Shipped	1	Smartphone	BrandX	699.99	1
2	2	2024-06-11 10:20:00	49.99	Processing	2	Jeans	BrandY	49.99	1
3	3	2024-06-11 15:45:00	19.99	Delivered	3	Cooking Book	BrandZ	19.99	1
4	4	2024-06-12 09:00:00	129.99	Pending	4	Microwave Oven	BrandA	129.99	1

Normalization

```
-- Create Location table (new)
CREATE TABLE Location (
    Location_id SERIAL PRIMARY KEY,
    State VARCHAR NOT NULL,
    City VARCHAR NOT NULL,
    Pincode INT NOT NULL
);

-- Create ReviewRating table (new)
CREATE TABLE ReviewRating (
    Review_id SERIAL PRIMARY KEY,
    Description VARCHAR,
    Rating_value INT CHECK (Rating_value BETWEEN 1 AND 10),
    Product_Product_id INT,
    Customer_Customer_id INT,
    FOREIGN KEY (Product_Product_id) REFERENCES Product(Product_id),
    FOREIGN KEY (Customer_Customer_id) REFERENCES Customer(Customer_id)
);

-- Create PaymentMode table (new)
CREATE TABLE PaymentMode (
    PaymentMode_id SERIAL PRIMARY KEY,
    PaymentMode_name VARCHAR NOT NULL
);
```